Implement the FastAPI Backend to Manage Routing and User Input Processing

Milestone 2 - Activity 2.2

1. Overview

Activity 2.2 is where the ComicCraft backend takes shape. Using **FastAPI**, the app defines the routes (URLs) users and API clients can call, captures the user's input, and connects each route to the AI functions built in Activity 2.1. All routing logic is written in **routes.py**. In short: this activity **defines the routes**, **processes user input** and **integrates the AI calls**.

2. Backend responsibilities

According to the architecture, the FastAPI backend manages all server-side work:

- Receive form data from the frontend, or JSON from API calls.
- Call the AI models to generate panel outlines, story narration and illustrations.
- Organize and build the complete comic layout.
- Export the comic to a downloadable PDF.
- Render the results into Jinja2 templates for preview and export.

The backend ensures proper coordination between user inputs, AI generation workflows and template rendering.

3. Define routes in FastAPI

Each route is linked to a backend function that runs part of the comic generation workflow from Activity 2.1. The routes handle:

- Form-based comic generation (from the browser).
- JSON-based comic generation (from API clients).
- Image testing.
- Comic export confirmation.

Route	Type	Purpose
`/`	Page	Loads the homepage (index.html) with the input form.
`/generate`	Form (POST)	Takes form input, runs the AI workflow, shows comic_preview.html.
`/generate-comic/json`	API (JSON)	Same flow, triggered by a raw JSON request; returns layout data and PDF path.
`/export-success`	Page	Confirms the comic export (export_success.html).
`/test-image`	Utility	Calls generate_image() directly with a custom prompt to test image generation separately.

4. Process user input

Step 1 - HTML form

The form is created in **index.html**. The user enters five details:

- Story prompt
- Main character name
- Setting
- Tone
- Art style

Step 2 - Capture inputs with Form(...)

FastAPI captures the submitted fields using `Form(...)` parameters in the route handler. The names of the parameters must match the `name` attributes of the HTML inputs. The `python-multipart` library (installed in Activity 1.3) is required for this.

Step 3 - Validate JSON with Pydantic

For API-based generation, a Pydantic schema called `PromptRequest` validates the JSON body. Wrong or missing fields are rejected automatically with a clear error, before your logic runs.

Input source	How it is captured	Used by
HTML form	Form(...) parameters	/generate
JSON body	PromptRequest (Pydantic model)	/generate-comic/json
Direct prompt	Simple prompt value	/test-image

5. Integrate the AI function calls

Each route handler connects directly to the function modules created in Activity 2.1.

/generate

- `generate_outline()` - creates panel outlines from the user prompt.
- `generate_story()` - expands them into narration and dialogue.
- `generate_image()` - called for each panel to create comic-style images.
- `build_comic_layout()` - organizes the final layout.
- `save_pdf()` - compiles and exports the complete comic to PDF.

/generate-comic/json

- Similar flow to /generate, but triggered by a raw JSON API request.
- Returns a JSON response with the layout data and the PDF path, for API clients.

/test-image

- Calls generate_image() directly with a custom prompt, to test image generation on its own.

All AI-generated text and images are processed, cleaned and injected into the Jinja2 templates (**comic_preview.html** and **export_success.html**) for a smooth user experience.

6. Request flow

```
Browser form  --POST-->  /generate  --> Form(...) inputs
API client    --JSON-->  /generate-comic/json --> PromptRequest
                                  |
   generate_outline() -> generate_story() -> generate_image() per panel
                                  |
        build_comic_layout() -> save_pdf()
                                  |
      HTML (comic_preview.html)   or   JSON (layout + PDF path)
```

7. Illustrative example

A simplified **example** of the JSON route and the test route. Your real routes.py may use different names.

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel

router = APIRouter()

class PromptRequest(BaseModel):
    prompt: str
    character: str
    setting: str
    tone: str
    art_style: str

@router.post("/generate-comic/json")
async def generate_comic_json(data: PromptRequest):
    try:
        outline = generate_outline(data.prompt, data.character,
                                   data.setting, data.tone, data.art_style)
        story   = generate_story(outline)
        images  = [generate_image(p["image_prompt"]) for p in outline]
        layout  = build_comic_layout(images, story)
        pdf     = save_pdf(layout)
        return {"layout": layout, "pdf_path": pdf}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/test-image")
async def test_image(prompt: str):
    return {"image_path": generate_image(prompt)}
```

8. Testing the backend

- Start the server: `uvicorn app.main:app --reload`
- Open `http://127.0.0.1:8000` to use the form.
- Open `/docs` to try /generate-comic/json, /generate, /export-success and /test-image interactively.
- Use /test-image first to confirm image generation works before running the full pipeline.

9. Summary checklist

- All routes defined in routes.py and linked to backend functions.
- Form fields captured with Form(...); JSON validated with PromptRequest.
- Each route calls the correct AI functions from Activity 2.1.
- Outputs passed to Jinja2 templates or returned as JSON.